

DATACOPILOT

AI-Driven AutoML & Conversational Insights Engine

Submitted For: ABB Technical Challenge

Date: May 20, 2026

Evaluation Focus: Innovation, Technical Implementation, Industrial Relevance, Scalability & Robustness

Scope: End-to-End Firebase-Firestore Integrated AutoML Lifecycle

1. Executive Summary

DataCopilot is a production-ready, full-stack Automated Machine Learning (AutoML) platform that bridges high-fidelity statistical modeling with explainable AI and conversational natural language querying. Built on a serverless, document-oriented architecture, DataCopilot verifies user identity via Google Sign-In, stores per-user sessions in Google Cloud Firestore, executes parallel model training workflows locally on FastAPI background threads, and streams step-by-step progress directly to a Next.js front-end using real-time WebSockets. This setup removes architectural complexity (no SQL database is required) while providing stateful, multi-user workspace isolation.

2. Inputs Considered

The DataCopilot pipeline accepts specific data, system config, and credential files to run the automated workflow and secure downstream user transactions:

2.1 Data and Schema Inputs

- **Structured Tabular Datasets:** Standard Comma-Separated (.csv) or JavaScript Object Notation (.json) files uploaded via multipart POST requests.
- **Schema Properties:** User-defined target column headers, or an inferred target keyword parsed from raw headers (e.g. *target*, *label*, *class*, *price*, *churn*).
- **Temporal Columns:** Date/time columns detected automatically by scanning columns for datetime types or headers matching temporal keywords (e.g. *date*, *time*, *timestamp*).
- **User Session UUID:** Unique identifiers generated per dataset upload to organize Firestore records and segment workspace cache directories.

2.2 Authentication, Credentials & System Configs

- **Firebase Admin Key Store:** Private JSON service account certificate (serviceAccountKey.json) loaded on the server to authenticate and authorize Firestore operations.
- **Bearer Token Header:** A cryptographically signed Firebase ID token (JWT) passed in the HTTP header (Authorization: Bearer <token>) to authorize protected REST/WS endpoints.
- **Language Model API Keys:** Private keys for Google Gemini Pro (main reasoning agent) and Groq Llama-3-70b (failover engine) stored in server-side .env configurations.
- **Firebase Client Environment Variables:** Web API configuration details (apiKey, authDomain, projectId) used to initialize client-side Google Sign-In.

3. Core AutoML Process to be Followed

DataCopilot executes an orchestrated 9-step pipeline to transform raw, noisy datasets into fully evaluated models backed by explainable artificial intelligence. The flowchart of operations spans data ingestion, statistical analysis, preprocessing, training, evaluation, and explanation.

Step 1: Upload & Auth	Step 2: Profile & Stats	Step 3: Task & Imbalance
FastAPI intercepts and verifies Bearer JWT tokens via Firebase Admin. Saves files and creates Firestore doc: users/{uid}/sessions/{session_id}.	Loads file in Pandas to compute null ratios, categorical frequencies, duplicate counts, and descriptive numeric distributions.	Detects target and temporal columns; infers ML task type (Classification vs. Regression vs. Clustering). Flags class imbalance (>75%).

Phase 1: Ingestion & Firebase Authentication Gate

Every protected request is intercepted by FastAPI middleware (auth_middleware.py), which extracts the Bearer token and verifies it against the Firebase Admin SDK, yielding the user's unique uid. The endpoint saves the raw file to backend/workspace/uploads/ and creates a stateful session record in Firestore. This architecture links database ownership to the authenticated user. A WebSocket endpoint is established (ws/analysis/{session_id}) to stream subsequent task outputs in real-time.

Phase 2: Comprehensive Profiling & Diagnostics

Once files are saved, a background worker loads the dataset using Pandas. It computes missing value ratios, duplicates, data types, and core statistics (mean, standard deviation, and quantiles for numeric columns; top-5 category distributions for objects). The resulting profile metadata is serialized, saved to the Firestore session record, and emitted via WebSocket to drive the frontend UI.

Phase 3: Task Classification & Imbalance Engineering

DataCopilot determines the machine learning objective through automated classification heuristics:

- Target Auto-Detection:** The system scans column names for target keywords. If none are found and no target is selected, it defaults to an unsupervised **Clustering** task.
- Temporal Detection:** Scanning for date/time keywords flags the session for **Time Series** forecasting.
- Task Selection:** For standard supervised datasets, the system checks target cardinality. Numeric targets with > 15 unique values are classified as **Regression**; target values with ≤ 15 values are classified as **Binary** or **Multi-class Classification**.
- Imbalance Checks:** For classification tasks, the system computes the majority class ratio. If any single class represents > 75% of the dataset, it sets an is_imbalanced flag, which triggers class-weight balancing configurations in downstream training.

3. Core AutoML Process to be Followed (Continued)

Following task identification, the pipeline processes the data to make it compatible with machine learning algorithms and selects optimal candidate models for competitive cross-validation.

Step 4: Preprocessing	Step 5: Recommend Models	Step 6: Threaded CV Train
Prunes sparse columns (>75% null). Imputes median/mode, encodes categories, and scales features via StandardScaler.	Recommends a list of candidate algorithms (XGBoost, RandomForest, MLP) custom-suited to the detected task.	Runs Stratified K-Fold or Time Series splits locally on server CPU threads. Streams progress updates fold-by-fold.

Phase 4: Automated Preprocessing & Feature Engineering

To prepare raw tabular features for scikit-learn and XGBoost pipelines, the engine applies transformations:

- Column Pruning:** Columns with more than 75% missing data are pruned to reduce feature noise.
- Imputation:** Missing continuous values are imputed with column medians; missing categorical variables are imputed with column modes. Imputation constants are cached for inference reproducibility.
- Feature Encoding:** Continuous variables are standardized (zero mean, unit variance) via StandardScaler. Categorical variables are One-Hot Encoded if they have ≤ 10 unique levels; otherwise, they are Label Encoded.

Phase 5: Candidate Model Recommendations

Instead of training a single model, DataCopilot defines a tailored search space based on the inferred task, recommending candidate architectures with detailed technical parameters:

Task Type	Candidate Algorithms & Parameter Justifications
Classification	XGBoost Classifier (gradient boosting), Random Forest (bagging), Logistic Regression, Naive Bayes (fast baseline), MLP Neural Network (non-linear representations)
Regression	XGBoost Regressor, Random Forest Regressor, ElasticNet (regularized linear), MLP Regressor
Clustering	K-Means (centroid-based), DBSCAN (density-based), Agglomerative Clustering (hierarchical linkage)

Phase 6: Threaded Training & Cross-Validation

All machine learning models are trained locally on the server CPU. Because training is computationally intensive, running it directly in FastAPI's async loop would block other WebSocket events and lock user pages. To prevent this, the training pipeline runs inside a background thread pool executor (`run_in_executor`). Overfitting is minimized through cross-validation splits ($K = \min(5, \max(2, N/30))$). The system uses StratifiedKFold for classification (to preserve imbalanced target bounds) and TimeSeriesSplit for temporal forecasting. Progress and training logs are pushed live via WebSocket fold-by-fold.

3. Core AutoML Process to be Followed (Continued)

The final phases of the pipeline compute diagnostic evaluation metrics, explain the underlying model behavior using SHAP values, and initialize the conversational LLM interface.

Step 7: Evaluate Test Set	Step 8: SHAP Explainability	Step 9: LangChain Chat
Tests models on a 20% holdout split. Computes Accuracy, F1, ROC-AUC, R ² , Silhouette, and Confusion Matrices.	Attributes predictions using SHAP values. Implements multi-tier fallbacks (tree, kernel, importances, coefficients).	Spawns a conversational agent in LangChain. Reads session variables from Firestore, calling Gemini or Groq APIs.

Phase 7: Model Evaluation & Holdout Testing

A separate 20% holdout test dataset is set aside to evaluate model generalization. The backend trains each candidate model on the training fold, predicts on the test set, and calculates a wide range of metrics: Classification models compute Accuracy, Precision, Recall, F1-Score, and ROC-AUC; Regression models calculate R², MAE, RMSE, and MAPE; Clustering models calculate Silhouette Scores and the Davies-Bouldin Index. For classification, the system also computes a complete Confusion Matrix and ROC curve coordinates for the best-performing model (based on CV score).

Phase 8: SHAP Interpretability & Feature Attribution

To provide model transparency, DataCopilot computes SHAP (SHapley Additive exPlanations) values for the best-performing model. Because SHAP calculations can be computationally intensive, the backend features a robust, multi-tier fallback architecture to guarantee zero-crash execution:

- TreeExplainer:** Activated for tree-based ensemble models (XGBoost, Random Forest) to compute exact SHAP values quickly.
- KernelExplainer:** Activated for non-tree models (KNN, MLP), sampling a small background dataset (up to 50 rows) to estimate feature impacts.
- Feature Importances Fallback:** Extracted directly using the model's native `feature_importances_` if explainer calculations fail.
- Linear Coefficients Fallback:** Extracted by averaging absolute model weights (`coef_`) if a linear baseline is chosen.

Phase 9: Conversational AI & LangChain Agent Context

Upon pipeline completion, the backend compiles the data profile, model scores, and SHAP features into a structured payload. This is passed to a background thread to generate an initial AI analysis using Google Gemini or Groq. The prompt instructs the LLM to explain the model choice, explain which features drive performance, point out any data anomalies, and offer business recommendations. This response is saved in Firestore as the session's `ai_insight` and sent to the client. Users can then ask follow-up questions in the chat interface, which is powered by LangChain and retrieves past messages from Firestore to maintain conversation context.

4. Expected Outputs

The DataCopilot system generates two primary types of outputs: a structured database state containing detailed machine learning metadata, and an interactive frontend dashboard UI.

4.1 Structured Firestore Session Schema (Per-User Storage)

The ultimate output of the pipeline is a rich JSON document persisted in Firestore under a unique session_id. This structured record enables full reproducibility and is defined as follows:

```
{
  "uid": "firebase_user_uid", "session_id": "823e6659-693b-491a-a05e-d5d9b11fad00",
  "filename": "customer_churn.csv", "status": "done", "created_at": "2026-05-20T02:12:14Z",
  "ai_insight": "The best model is XGBoost (94.2% Acc)... Key Drivers: MonthlyCharges, Tenure...",
  "steps": {
    "upload": { "status": "done", "data": { "rows": 7043, "cols": 21 } },
    "analyze": { "status": "done", "data": { "per_column": [...], "duplicate_rows": 0 } },
    "task": { "status": "done", "data": { "target_col": "Churn", "task_type": "Binary Classification" } },
    "preprocess": { "status": "done", "data": { "transforms": [...], "feature_names": [...] } },
    "recommend": { "status": "done", "data": { "models": [...] } },
    "train": { "status": "done", "data": { "cv_results": { "xgboost": { "mean": 0.942, ... } } } },
    "evaluate": { "status": "done", "data": { "metrics": [...], "confusion_matrix": [...] } },
    "shap": { "status": "done", "data": { "features": [ { "name": "Tenure", "importance": 0.354 } ] } },
    "viz": { "status": "done", "data": { "correlation_matrix": [...], "histograms": {...} } }
  }
}
```

4.2 Next.js Dashboard Mockup & Client Contexts

The Next.js frontend uses state contexts to manage and render a premium, responsive dashboard: AuthContext maintains user states and injects JWT authorization headers, while SessionContext manages the active WebSocket connection. Real-time metrics are rendered in a sleek dark-themed dashboard:

